# Practical: NumPy – Elements of Similar Data Types

## 1. One-Dimensional Array

**Input Code**

```
import numpy as np

# 1D array
a = np.array([1, 2, 3])
print(type(a))
print(a.shape)
print(a[0], a[1], a[2])
a[0] = 5
print(a)
```

**Output**

```
<class 'numpy.ndarray'>
(3,)
1 2 3
[5 2 3]
```

## 2. Array Filled with Zeros

**Input Code**

```
import numpy as np
zeros_array = np.zeros((3, 3))
print(zeros_array)
```

**Output**

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

## 3. Array Filled with Ones

**Input Code**

```
import numpy as np
ones_array = np.ones((3, 2))
print(ones_array)
```

**Output**

```
[[1. 1.]
 [1. 1.]
 [1. 1.]]
```

## 4. Array Using arange()

**Input Code**

```
import numpy as np
range_array = np.arange(0, 10, 2)
print(range_array)
```

**Output**

```
[0 2 4 6 8]
```

## 5. Evenly Spaced Values Using linspace()

**Input Code**

```
import numpy as np
linear_array = np.linspace(0, 1, 5)
print(linear_array)
```

**Output**

```
[0.   0.25 0.5  0.75 1.  ]
```

## 6. Two-Dimensional Array

**Input Code**

```
# 2D Array
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b.shape)
print(b[0, 0], b[0, 1], b[1, 0])
print(b)
```

**Output**

```
(2, 3)
1 2 4
[[1 2 3]
 [4 5 6]]
```

## 7. Random Array

### Input Code

```
import numpy as np
random_array = np.random.rand(2, 3)
print(random_array)
```

### Output

```
[[0.10016543 0.19571958 0.72663888]
 [0.91028117 0.02314189 0.77672876]]
```

## 8. NumPy Array Operations

### Input Code

```
arr1 = np.array([[4, 7], [2, 6]], dtype=np.float64)
arr2 = np.array([[3, 6], [2, 8]], dtype=np.float64)
print(arr1)
print(arr2)
print(np.add(arr1, arr2))
print(np.sum(arr1))
print(np.sqrt(arr1))
print(arr1.T)
```

### Output

```
[[4. 7.]
 [2. 6.]]
[[3. 6.]
 [2. 8.]]
[[ 7. 13.]
 [ 4. 14.]]
19.0
[[2.         2.64575131]
 [1.41421356 2.44948974]]
[[4. 2.]
 [7. 6.]]
```

## 9. Addition, Subtraction, Multiplication and Division

### Input Code

```
import numpy as np

x = np.array([[1, 2], [3, 4]], dtype=np.float64)
y = np.array([[5, 6], [7, 8]], dtype=np.float64)

print(x + y)
print(np.add(x, y))
print()

print(x - y)
print(np.subtract(x, y))
print()

print(x * y)
print(np.multiply(x, y))
print()

print(x / y)
print(np.divide(x, y))
print()

print(np.sqrt(x))
```

### Output

```
[[ 6.  8.]
 [10. 12.]]
[[ 6.  8.]
 [10. 12.]]

[[-4. -4.]
 [-4. -4.]]
[[-4. -4.]
 [-4. -4.]]
```

```
[[ 5. 12.]
 [21. 32.]]
[[ 5. 12.]
 [21. 32.]]

[[0.2        0.33333333]
 [0.42857143 0.5       ]]
[[0.2        0.33333333]
 [0.42857143 0.5       ]]

[[1.         1.41421356]
 [1.73205081 2.        ]]
```

## 10. Dot Product of Vectors

### Input Code

```
import numpy as np

x = np.array([[1, 2], [3, 4]])
y = np.array([[5, 6], [7, 8]])
print(x)
print(y)

v = np.array([9, 10])
w = np.array([11, 12])
print(v)
print(w)

# Inner product of vectors
print(v.dot(w))
print(np.dot(v, w))
print(np.multiply(v, w))
```

### Output

```
[[1 2]
 [3 4]]
[[5 6]
 [7 8]]
[ 9 10]
[11 12]
219
219
[ 99 120]
```

## 11. Dot Product of Matrix and Vector

### Input Code

```
print(x.dot(v))
print(np.dot(x, v))
```

### Output

```
[29 67]
```

## 12. Mean of Array

### Input Code

```
import numpy as np
array = np.array([1, 2, 3])
mean_value = np.mean(array)
print(mean_value)
```

### Output

```
2.0
```

## 13. Maximum and Minimum Values

### Input Code

```
import numpy as np
array = np.array([1, 2, 3])
print(array)
max_val = np.max(array)
min_val = np.min(array)
print(max_val)
print(min_val)
```

### Output

```
[1 2 3]
3
```

```
1
```

## 14. Array Indexing (Slicing)

### Input Code

```
import numpy as np
array = np.array([1, 2, 3, 4])
element = array[2]
print(element)
```

### Output

```
3
```

## 15. Array Slicing

### Input Code

```
import numpy as np
subset = array[1:3]
print(subset)
```

### Output

```
[2 3]
```

## 16. Sorting an Array

### Input Code

```
import numpy as np
sorted_array = np.sort(np.array([3, 1, 2]))
print(sorted_array)
```

### Output

```
[1 2 3]
```

## 17. Using np.where()

### Input Code

```
# np.where(): Returns the indices of elements that satisfy a condition.
import numpy as np
array = np.array([1, 2])
indices = np.where(array > 2)
print(indices)
```

### Output

```
(array([], dtype=int64),)
```